

JAVA INTERVIEW PREPARATION

CHAPTER 42

MICROSERVICE BOUNDARIES AND
DISTRIBUTED-SYSTEM FUNDAMENTALS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Microservice Boundaries and Distributed-System Fundamentals: Designing for Independent Change

Senior / Lead Java Developer Reading Chapter

A microservice is not simply a Spring Boot application with an HTTP controller. It is an independently operated capability with explicit ownership of behavior and data. The reason to introduce a service boundary is usually organizational and evolutionary: one team must be able to change, deploy, scale, and recover a capability without coordinating every release with the entire system.

Distribution has a permanent price. A method call becomes a network exchange that can be slow, duplicated, reordered, partially completed, or unavailable. A local transaction becomes a workflow across separate consistency boundaries. Debugging requires correlation across processes. If independent change does not repay those costs, a well-structured modular monolith is often the stronger design.

Start with Business Capabilities

A useful boundary groups rules and data that change together. Order placement, catalog management, payment authorization, and shipment planning are capabilities with different language, invariants, load patterns, and operational risk. Splitting by technical layer creates services such as `controller-service`, `business-service`, and `database-service`; nearly every feature then crosses every service, so the split increases latency without creating autonomy.

```
Weak technical split:
UI service -> logic service -> data service
every feature requires all three teams and deployments

Capability split:
Order service      owns order lifecycle and order data
Payment service    owns authorization and payment records
Shipping service   owns shipment planning and carrier state
```

Domain-driven design calls a coherent language and model a bounded context. The same word can legitimately mean different things across contexts. A `Customer` in billing may mean payer identity and credit terms; in shipping it may mean recipient address and delivery preference. Sharing one universal customer object couples both models and makes every field appear globally authoritative.

The boundary should contain invariants that require immediate consistency. If confirming an order requires checking and updating several pieces of order state atomically, keep that state under one transactional owner. A design that repeatedly requires distributed transactions to preserve a basic invariant is often signaling the wrong boundary.

A Modular Monolith Is a Valid Architecture

A modular monolith deploys as one process but enforces internal module boundaries. It offers local calls, one operational unit, simpler debugging, and straightforward transactions while preserving conceptual ownership.

```
one deployable
order module    -> public application API + private implementation
payment module  -> public application API + private implementation
shipping module -> public application API + private implementation
```

This can be the best starting point when the domain is still being discovered, the team is small, load does not require independent scaling, or deployment coordination is not a bottleneck. Clear modules create an extraction path later. A tangled monolith cannot be repaired merely by putting network calls between its packages; that produces a distributed monolith.

Extract a service when evidence supports it: a capability needs a separate release cadence, stronger isolation, distinct regulatory control, independently dominant scale, or a team can truly own it end to end. Do not extract because microservices are associated with senior architecture.

Service Ownership Includes Data

A service owns the schema and interpretation of its data. Other services should use its contract rather than querying its tables. Direct cross-service reads create hidden coupling to schema names, migration timing, soft-delete rules, and tenant policy. They also let one service bypass the owner's authorization and invariants.

```
Order Service           Payment Service
order database          payment database
X----- no direct joins -----X
    API, event, or replicated read model
```

Database-per-service means logical ownership, not necessarily one physical database server for every small service. Separate schemas or credentials can be an intermediate step if access controls prevent cross-ownership queries. The critical property is that only the owning service writes and interprets its data contract.

Cross-capability views require composition. A request can query several services, or events can build a local read model optimized for the view. Synchronous composition gives fresher answers but adds latency and availability dependencies. A replicated view serves quickly and independently but is eventually consistent. The product requirement determines which tradeoff is correct.

Distributed Failure Is Partial

Inside one process, a call normally returns or throws while both caller and callee share the same fate. Across a network, the caller may time out without knowing whether the server received or committed the operation.

```
caller sends payment request
-> payment commits authorization
-> response is lost
caller sees timeout: outcome is UNKNOWN, not necessarily FAILED
```

This ambiguity is central to distributed design. Retrying may duplicate the effect; not retrying may lose a required operation. Idempotency keys, operation status resources, deduplication, and reconciliation exist because a timeout does not reveal the remote truth.

Failures also correlate. Ten downstream calls do not fail independently when they share a database, region, certificate, DNS resolver, or thread pool. Availability multiplication is a useful warning but not a complete model. Mapping shared dependencies and failure domains is more important than quoting a theoretical percentage.

Latency Compounds Across Call Graphs

A user request that fans out or passes through many sequential services inherits queueing and tail latency from the graph.

```
API -> Order -> Pricing -> Promotion -> Customer
      \-> Inventory
```

Sequential calls add latency. Parallel calls reduce the critical path only when they are independent, but increase simultaneous load and require an executor or non-blocking model with explicit ownership. A fan-out of 100 calls makes one slow or failed dependency statistically common and can overwhelm the caller's connection pool.

Set an end-to-end latency budget, then allocate deadlines to downstream work while reserving time to serialize and respond. A local timeout larger than the caller's remaining deadline wastes resources after the answer can no longer be used. Queue time is part of latency; a request that spends 900 ms waiting and 20 ms executing still violated a 500 ms objective.

Contracts Between Independently Deployed Services

Service contracts include payloads, headers, error meanings, timeouts, retry rules, ordering, authentication, and compatibility. An OpenAPI or event schema documents structure, but it does not by itself define operational semantics.

Prefer tolerant, additive evolution. A provider can add optional response fields when consumers tolerate unknown data. Removing a field, changing its meaning, or adding a required request property needs a migration. Deploy consumers that understand both old and new forms before providers stop producing the old form.

```
expand -> migrate -> contract

1. provider accepts old and new
2. consumers move to new
3. telemetry confirms old is unused
4. provider removes old after policy permits
```

Consumer-driven contracts can catch incompatible assumptions, but the provider must retain governance. A contract test should not permanently freeze an accidental behavior. Define which versions are supported, how deprecation is communicated, and who owns consumer migration.

Coupling Beyond the Network Contract

Services can remain tightly coupled even with clean JSON. Temporal coupling requires both services to be available at the same moment. Deployment coupling requires coordinated releases. Semantic coupling occurs when consumers reproduce the provider's rules. Operational coupling occurs when services share one saturated database, cluster, certificate, or on-call team.

Shared Java libraries deserve particular care. A small library for trace propagation or stable error primitives can reduce inconsistency. A shared library containing domain entities, persistence mappings, and business services creates binary coupling and coordinated upgrades. Prefer generated or independently versioned contract types at external boundaries, and let each service maintain its internal model.

Measure coupling through change history: how many repositories, teams, pipelines, schemas, and production approvals must move for one feature? If every change requires a synchronized release train, the system is not independently deployable regardless of how many processes it contains.

Identity and Trust Between Services

Internal traffic is not automatically trusted. Authenticate workloads through platform identity, mutual TLS, signed tokens, or another maintained mechanism. Authorize the calling service and, when required, propagate the end-user identity with clear delegation semantics.

```
external user -> gateway -> Order Service -> Payment Service
                user identity      workload identity + delegated user context
```

Payment must know whether it trusts Order to act on behalf of this user and for which scopes. Forwarding the original bearer token everywhere expands its audience and blast radius. Token exchange or audience-specific credentials can preserve least privilege.

Network policy limits reachable paths but does not replace application authorization. Encrypt traffic according to the threat model, rotate workload credentials, and audit privileged service-to-service actions. Trace and baggage headers are context, not proof of identity; accept security identity only from verified authentication mechanisms.

Service Discovery and Load Balancing

A logical service name must resolve to healthy instances. In Kubernetes, platform DNS and Services commonly provide discovery and load balancing. Other environments may use a registry or a gateway. Do not add an application-level discovery system when the platform already owns this responsibility.

Discovery tells the client where an instance might be; it does not guarantee that the selected instance can successfully handle the operation. Readiness, connection draining, retry policy, and load balancing still matter. DNS caches and keep-alive connections can retain old endpoints during deployment, so termination grace must account for propagation.

Client-side load balancing gives the caller instance-level choice and metrics, but distributes policy across clients. A proxy centralizes policy and supports non-Java clients, but adds a hop and another operational component. Service meshes can provide transport identity, retries, and telemetry, yet application semantics such as idempotency and fallback correctness remain in application ownership.

API Gateways and Backend-for-Frontend

An edge gateway can terminate TLS, authenticate external clients, apply coarse rate limits, route requests, and enforce common protocol policy. It should not become the location of every business rule. A "smart gateway, dumb services" design recreates a central monolith at the most critical network boundary.

A backend-for-frontend tailors composition to a particular client, such as mobile or web. It can reduce chatty external calls and shield clients from internal topology. Domain decisions remain with the owning services, and object authorization must not be trusted solely to the edge because internal paths may exist.

Gateways also affect failure semantics. Their timeouts must be longer than intentional service deadlines but shorter than client abandonment. They must preserve or generate trace context safely, limit request sizes, and avoid automatically retrying non-idempotent writes.

Configuration and Secrets

Configuration varies by environment while code remains immutable. Validate mandatory configuration at startup and record non-secret effective origins for diagnosis. A central configuration service can coordinate change, but it also becomes a dependency and potential blast radius.

Secrets need controlled storage, access, rotation, and audit. Do not place credentials in images, source repositories, logs, exception bodies, Actuator output, or trace baggage. Rotation must account for client connection pools and cached credentials; changing a secret in a vault does not automatically rebuild every live client.

Dynamic configuration introduces consistency questions. Instances may observe different values during rollout. For safety-critical policy, prefer versioned configuration and explicit activation. Feature flags need

ownership, expiry, and behavior when the flag service is unavailable.

Deployment and Compatibility

Independent deployability requires more than separate pipelines. A new service version must coexist with old callers, old database records, and sometimes old instances during rolling deployment. Database migrations should use expand-and-contract: add compatible structures, deploy code that can use both, migrate data, then remove old structures later.

Canary releases direct a small portion of representative traffic to a new version and compare service-level indicators. Blue-green deployment switches between complete environments. Neither automatically makes database or event changes reversible. A rollback may fail if the new version wrote data the old version cannot read.

Version, configuration identity, build commit, and schema migration state should be observable. During an incident, "the same code" is insufficient if instances use different flags, secrets, or dependency endpoints.

Observability Across Boundaries

Each service should expose metrics for traffic, errors, latency, saturation, and domain outcomes. Distributed traces show the path and remaining latency budget; structured logs retain detailed events. Correlation must propagate through HTTP and messaging using supported standards.

Use route templates rather than raw IDs in metric labels. Treat baggage as untrusted input, restrict size and keys, and never propagate credentials. Sampling means traces cannot replace complete metrics. A trace explains an example; metrics establish whether the behavior is widespread.

Service maps are useful, but dependency ownership and criticality should also be documented. A dashboard can show that Order calls Pricing; it does not explain whether cached pricing is an approved degraded mode or whether failure must reject the order.

Testing Service Boundaries

Test the domain and application logic locally, contract-test provider and consumer assumptions, and integration-test real serialization, authentication, timeout, and error behavior. A mock server that always returns perfect responses cannot prove compatibility with the real provider.

Resilience tests should inject specific failures: slow connection, slow body, reset after request commit, malformed response, rate limit, expired certificate, and partial dependency outage. Broad "chaos" without a hypothesis produces noise. State the expected invariant and degraded behavior first, then verify it.

End-to-end tests are expensive and should cover a small number of business-critical journeys. They cannot enumerate all combinations in a large service graph. Ownership remains local: each service proves its contract and publishes enough evidence for system-level verification.

Common Senior-Level Traps

The first trap is defining a service per database table or entity. This produces chatty workflows and distributed invariants. The second is sharing one database and calling the system microservices; schema coupling then prevents independent change. The third is assuming a timeout means the remote operation failed.

Other traps include deep synchronous call chains, universal shared libraries that force coordinated upgrades, central gateways containing domain logic, events that expose internal entity schemas, and dozens of tiny services owned by the same small team. Operational maturity and team topology are architectural constraints.

A Strong Interview Answer

I use a microservice boundary when a cohesive business capability benefits from independent ownership, deployment, scaling, or isolation enough to repay distributed-systems cost. I keep invariants and authoritative data under one owner, forbid direct cross-service table access, and integrate through explicit versioned contracts. I assume networks create ambiguous outcomes, correlated failures, and tail latency, so every remote call has a deadline, identity, observability, and defined retry semantics. Cross-service views choose deliberately between synchronous freshness and event-built local read models. I design deployments for coexistence, configuration and secret rotation, and evidence-based rollback. If those benefits are not present, I prefer a modular monolith with enforceable module boundaries.

Interview-Ready Summary

Microservices optimize independent change, not code size. Boundaries should follow business capabilities and contain strongly consistent invariants. Data ownership forbids cross-service schema coupling, while API or event composition accepts explicit freshness and availability tradeoffs.

Networks introduce ambiguous outcomes, compounded latency, and correlated failure. Contracts therefore include operational semantics, compatibility, security, and observability. Discovery, gateways, meshes, and deployment platforms provide mechanisms but do not remove application responsibility for idempotency, authorization, fallback correctness, and data evolution.

Revision Notes

- A microservice is an independently operated business capability.
- Distribution must be justified by independent change, scale, or isolation.
- A modular monolith is often the stronger starting point.
- Split by cohesive capability, not controller, service, and database layers.
- Keep immediately consistent invariants under one transactional owner.
- Service data ownership prohibits direct cross-service table access.
- Cross-service views choose freshness versus local availability deliberately.
- A timeout creates an unknown remote outcome, not proof of failure.
- Sequential calls add latency; fan-out adds load and tail risk.
- Allocate downstream deadlines from the end-to-end budget.
- Contracts include errors, retry semantics, identity, and compatibility.
- Use expand-migrate-contract for compatible evolution.
- Discovery locates instances but does not guarantee successful handling.
- Gateways own edge protocol policy, not every domain rule.
- A service mesh does not make unsafe retries or fallbacks correct.
- Validate configuration and design secret rotation lifecycle.
- Rolling deployments require old and new versions to coexist.
- Rollback may be unsafe after incompatible data has been written.
- Metrics detect widespread impact; traces explain representative requests.
- Test explicit failure modes and degraded invariants at service boundaries.